BDH Strategic Masking & Classification - Detailed Technical Design

Document Control

Document Status

Date	Version	Status
24-Feb-2026	v.0	

Document Authors

Name	Role
Naresh Kala	Platform Team Member
Syed Fayaz	Onshore Lead
Pavan Jangam	Platform Team Member

Endorsers and Approvers

Name	Role	Purpose	Version	Date	Status

Key Revision History

Version	Date	Description	Author(s)	Status
0.1	24/02/2026	First Draft	Naresh Kala	Draft
0.2	3/10/2026	Modified the review comments	Naresh Kala	Updated
0.3	3/13/2026	Reviewed	Syed Fayaz	Reviewed
0.4	3/16/2026	Reviewed	Daniel Dove	Reviewed

Summary

The architecture ensures that classification metadata extracted from Alation flows through a governed dbt pipeline into the Snowflake Governance Database, where it drives automated tag application and masking policy enforcement across all data product tables.

Overview

This document provides a high-level design for automating the ingestion of classification metadata from Alation and applying tagging and masking policies across Snowflake data product tables.

The solution removes manual classification steps and enforces consistent data protection through a governed dbt pipeline.

Objectives

- Automatically ingest classification metadata from Alation via Snowflake Data Share
- Apply Snowflake tags and masking policies without manual intervention
- Centralize all classification records in a single Governance Database
- Orchestrate and schedule the end-to-end pipeline through Airflow
- Maintain full auditability of tagging and masking actions

Current State vs Target State

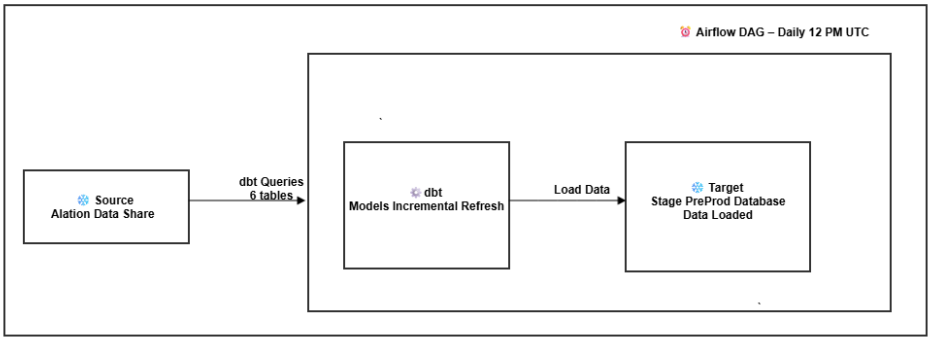
Aspect	Current State (As-Is)	Target State (To-Be)
Classification Entry	Manual entries into CLASSIFICATION_DATA	Automated ingestion from Alation Data Share

Tag Application	dbt macros trigger on schedule	dbt macros trigger on schedule
Masking	Applied via tags	Applied via tags
Processing Scope	Full refresh each run	Incremental - only new or changed records processed
Scheduling	Ad hoc	Daily automated run via Airflow

High-Level Architecture

Classification metadata originates in Alation and flows through a Snowflake Database .

Pipeline 1 | Snowflake(Alation data share) BDH-Snowflake PreProd Stage Database



1. Source - Snowflake (Alation Data Share)
The source system is Snowflake, with data exposed via Alation Data Share. Alation acts as the data catalog and governance layer on top of Snowflake, providing controlled access to 6 source tables. dbt connects directly to this Snowflake source to query the data.

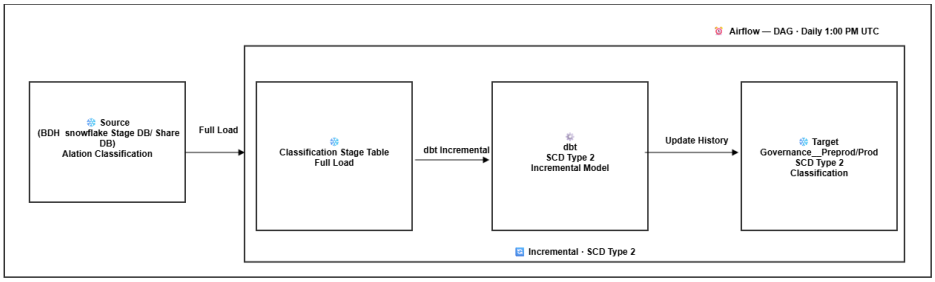
2.dbt - Models
dbt queries all 6 tables from the Snowflake Alation Data Share. It runs transformation models in Incremental refresh mode ,on every execution, Incremental logic is applied. The output is written directly into the Snowflake Stage database in preprod environment.

3.Target - Snowflake Preprod Stage DB
The target is a Snowflake preprod stage database. After dbt completes its run, all 6 tables are refreshed and available in the staging layer for downstream consumption.

Astro Airflow - Orchestration
Airflow orchestrates the entire pipeline via a DAG triggered daily at 12:00 PM UTC. Airflow manages task execution order, monitors success or failure, and triggers the dbt job. If any task fails, Airflow halts downstream steps and raises an alert. Both dbt and the Snowflake connection are managed within the Airflow DAG.

Note : **Airflow dag will be excute using Governance preprod Astro deployment workspace.**

Pipeline 2 | BDH-Snowflake(Preprod Stage DB/Prod Share DB) Governance__Preprod/Prod



1.Source - BDH Snowflake preprod Staging DB/Share DB
The source for this pipeline is the BDH Snowflake Stage DB/Share DB populated by Pipeline 1. It contains the Alation classification data that needs to be tracked and governed. This acts as the input for the daily classification load.
2.Stage Table - Full Load
The first step loads the filtered Alation classification data(6 tables queries) as single combined result ingested into **single** classification staging table in Snowflake on every run. This is a full load — the stage table is fully truncated and reloaded each time.
3.dbt - SCD Type 2 Incremental Model
dbt reads from the stage table and applies an incremental SCD Type 2 model. It compares incoming records against the Governance__preprod/prod DB, identifies new or changed records, and inserts them with effective date tracking- preserving full history. Unchanged records are not re-processed.
4.Target - Governance__Preprod/Prod (SCD Type 2)
The Governance__preprod/prod DB stores the Alation classification data with full history using SCD Type 2. Each record tracks when it became active and when it was superseded, enabling historical reporting and audit of classification changes over time.

SCD Type 2 — How it works in this pipeline

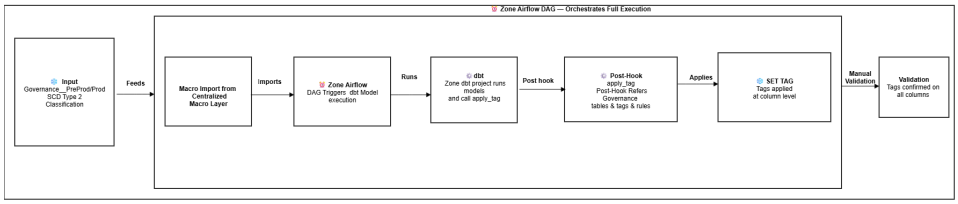
dbt incremental model compares the full stage table snapshot against the Governance__preprod/prod DB on each run:

Scenario	Action in Governance DB
New classification record	INSERT new row - effective_start_date = today, is_active = TRUE
Existing record changed	UPDATE old row (effective_end_date = today, is_active = FALSE) + INSERT new row
Record unchanged	No action - row skipped

Astro Airflow — Orchestration
Airflow DAG triggers daily at 1:00 PM UTC. It runs after Pipeline 1 (12:00 PM UTC) ensuring the Snowflake share DB/Share Prod DB is already refreshed before Pipeline 2 begins.

Note : Airflow dag will be excute using Governance preprod/prod Astro deployment workspace.

Pipeline 3 | Apply Tag Macro Flow Zone Airflow DAG to Column Classification & Validation



1.Governance__Preprod/Prod DB (Pipeline 2 SCD Type 2) — Input
The Governance__Preprod/Prod DB, populated by Pipeline 2 using SCD Type 2 incremental load, provides the classification rules. It is the reference source that the post-hook queries to determine which tags to apply to each column.
2. Macro Imported via packages.yml · dbt deps
The zone dbt project imports the apply_tag macro from the Centralized Macro Layer via packages.yml. Running dbt deps pulls the versioned macro, making it available as a post-hook across all models in the zone project.
3.Zone Airflow DAG -Triggers the Model Job
The Zone Airflow DAG initiates the full execution. It triggers the dbt model job in the zone dbt project, orchestrates task order, and manages success or failure alerting throughout the pipeline.
4.dbt Model Job
The zone dbt project runs its models. Each model materializes in Snowflake under the target schema following Domain__Layer__env naming convention (e.g. Cust__Foundation__dev).
5.Post-Hook Runs — Refers Governance DB Tables & Tags
After each model materializes, the **scm_apply_tag** post-hook fires automatically. It queries the Governance DB to look up the classification rules, tags, and masking policies defined for that model's tables and columns.
6.SET TAG - Column Level Classification Applied
The macro issues ALTER TABLE … SET TAG DDL statements in Snowflake. Tags are applied at the column level — transparent to model SQL, enforced by Snowflake at query time based on CURRENT_ROLE().
7.Validation — End of Pipeline
Once the **Zone Airflow DAG** has completed execution of the model job and **scm_apply_tag** macro, manual validation step runs to confirm classification has been applied correctly across all columns.

Note:

As part of the Zone Automation process, Flyway executes against each Snowflake database and applies three tags at the database level: security_classification_code, anchor_tag, and zone_tag

Zone Automation (Flyway) sets at DATABASE level:
security_classification_code = 'Unclassified'
anchor_tag
zone_tag - set here, inherited by all below

All Schemas, Tables, Columns inherit these 3 tags automatically

dbt Macro(**scm_apply_tag)** only overrides at column level:
security_classification_code = 'Highly Confidential' / 'Confidential/ 'Private' .

business_key_flag, and primary_key_flag tag

(from CLASSIFICATION_DATA_Active after data ingestion from alation share)

zone_tag and anchor_tag are never touched by the macro they are inherited from database level and managed purely by zone automation.

Key Technical components

Database and Table Details

Strategic masking and classification , governance database details.

DB NAME: ALATION__ANALYTICS__SHARE **SCHEMA NAME**: PUBLIC	**TABLE NAMES:** ALATION_SET_MEMBER CATALOG_SET_MEMBERSHIP RDBMS_COLUMNS RDBMS_TABLES RDBMS_SCHEMAS RDBMS_DATASOURCES
In Preprod: **DB NAME**: GOVERNANCE__STG_ALATION__PREPROD **SCHEMA NAMES:** ALATION **In Prod** **DB NAME**: : BNZ_DAP_PREPROD_AWS__STGANALYTICS__SHARE **SCHEMA NAMES:** ALATION	**TABLE NAMES:** ALATION_SET_MEMBER CATALOG_SET_MEMBERSHIP RDBMS_COLUMNS RDBMS_TABLES RDBMS_SCHEMAS RDBMS_DATASOURCES
DB NAME: GOVERNANCE__<ENV> **SCHEMA NAMES:** GOVERNANCE_CLASSIFICATION GOVERNANCE_STAGING GOVERNANCE_MASKING GOVERNANCE_TAGGING GOVERNANCE_LOGGING <ENV>: Environment identifier (PREPROD/PROD)	**TAGS:(Schema :GOVERNANCE_TAGGING)** ANCHOR_TAG BUSINESS_KEY_FLAG PRIMARY_KEY_FLAG SECURITY_CLASSIFICATION_CODE ZONE_TAG **MASKING POLICIES:(schema :GOVERNANCE_MASKING)** SENSITIVE_DATA_MASK_BOOLEAN SENSITIVE_DATA_MASK_DATE SENSITIVE_DATA_MASK_NUMBER SENSITIVE_DATA_MASK_STRING SENSITIVE_DATA_MASK_TIMESTAMP **TABLE NAME(Schema :GOVERNANCE_STAGING)** GOVERNANCE_CLASSIFICATION_DATA **TABLE NAME(Schema :GOVERNANCE_CLASSIFICATION)** GOVERNANCE_CLASSIFICATION_DATA_HISTORY **VIEW NAME(Schema :GOVERNANCE_CLASSIFICATION)** CLASSIFICATION_DATA_ACTIVE **TABLE NAME(Schema :GOVERNANCE_LOGGING)** **<<TBD>>**

Load Strategy Classification

Tables are classified into two strategies based on their characteristic.

ALATION__ANALYTICS__SHARE BDH -SNOWFLAKE (STAGE DB/ SHARE DB)

Classification data is sourced from the Alation Analytics Share and ingested into the BDH Snowflake environment via the Governance Staging Database (GOVERNANCE_STG_ALATION_PREPROD). The following Alation tables are replicated into the staging layer to support the classification and tagging process.

ALATION__ANALYTICS__SHARE	GOVERNANCE__STG_ALATION__PREPROD	Load Strategy	Unique_Key	Filtered_Logic
PUBLIC. CATALOG_SET_MEMBERSHIP	ALATION.CATALOG_SET_MEMBERSHIP	Incremental (Watermark column : DIM_TS_CREATED)	CATALOG_SET_PR OPERTY_ID	
PUBLIC.ALATION_SET_MEMBER	ALATION.ALATION_SET_MEMBER	Incremental (Watermark column : DIM_TS_CREATED)	ID	
PUBLIC.RDBMS_TABLES	ALATION.RDBMS_TABLES	Incremental + Filtered (Watermark column : DIM_TS_CREATED)	TABLE_ID	DS_ID IN(19,27) 19 -snowflake Prod 27- snowflake PreProd
PUBLIC.RDBMS_SCHEMAS	ALATION.RDBMS_SCHEMAS	Incremental + Filtered (Watermark column : DIM_TS_CREATED)	SCHEMA_ID	DS_ID IN(19,27) 19 -snowflake Prod 27- snowflake PreProd
PUBLIC.RDBMS_COLUMNS	ALATION.RDBMS_COLUMNS	Incremental + Filtered (Watermark column : DIM_TS_CREATED)	COLUMN_ID	DS_ID IN(19,27) 19 -snowflake Prod 27- snowflake PreProd
PUBLIC.RDBMS_DATASOURCES	ALATION.RDBMS_DATASOURCES	Incremental + Filtered (Watermark column : DIM_TS_CREATED)	DS_ID	DS_ID IN(19,27) 19 -snowflake Prod 27- snowflake PreProd

GOVERNANCE__STG_ALATION__PREPROD - > GOVERNANCE__PREPROD/PROD

Once alation classification data is ingested into **governance_stg_alation_preprod**, all six tables are shared to the production environment through the snowflake share **bnz_dap_preprod_aws_stganalytics_share**.
these six shared tables are then joined and transformed within **governance__preprod/governance_prod** to produce a consolidated classification table **goverance_staging.governance_classification_data**.

GOVERNANCE__STG_ALATION__PREPROD -> BNZ_DAP_PREPROD_AWS__STGANALYTICS__SHARE ->	GOVERNANCE__PREPROD GOVERNANCE__PROD	Load Strategy
ALATION.ALATION_SET_MEMBER ALATION.CATALOG_SET_MEMBERSHIP ALATION.RDBMS_COLUMNS ALATION.RDBMS_TABLES ALATION.RDBMS_SCHEMAS ALATION.RDBMS_DATASOURCES	GOVERNANCE_STAGING. GOVERANACE_CLASSIFICATION_DATA	Full Load

GOVERNANCE__PREPROD/PROD - > GOVERNANCE__PREPROD/PROD

The consolidated staging table **goverance_staging.governance_classification_data** is incrementally loaded into **governance_classification.governance_classification_data_history**, maintaining a full history of classification changes over time to support delta-based column-level tagging

GOVERNANCE__PREPROD/PROD	GOVERNANCE__PREPROD/PROD	Load Strategy
GOVERNANCE_STAGING. GOVERANACE_CLASSIFICATION_DATA	GOVERNANCE_CLASSIFICATION. GOVERANACE_CLASSIFICATION_DATA_HISTORY	Incremental (Watermark column : **RECORD_CREATED_AT**)

Macro creation & deployment

Reusable dbt macros are created and managed within the **bdh_platform_dbt_repo** project, where they are independently version-controlled and tested in a centralized repository.

DBT Project Setup: Reusable Macros, Model Naming, and Execution Flow - Digital, Data & Analytics - BNZ Confluence

Macro Overview

The **scmapply_tags** macro it reads column-level classification data from classification_data_active and applies three tags security_classification_code, business_key_flag, and primary_key_flag directly onto the target snowflake table columns using alter table ... modify column set tag.

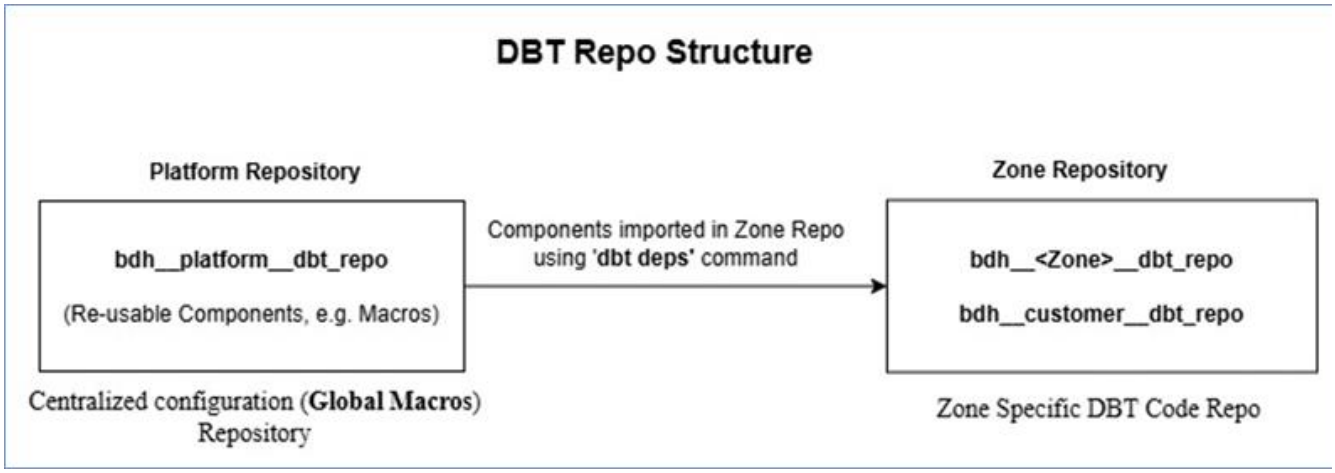
Key Macro Behaviors

- Idempotent — safe to re-run; will not create duplicate tags or policy assignments
- Incremental — processes only new or changed classification records
- Fully logged — all actions written to GOVERNANCE__PREPROD/PROD/GOVERNANCE_LOGGING for audit

Distribution & Import Overview

The **scmapply_tags** macro in **bdh_platform_dbt_repo** are packaged and published for reuse across dbt projects. Target projects import these macros via dbt's package system, enabling them to reference shared logic directly from the central library—eliminating the need for code duplication.

Projects integrate the shared macro library by specifying it in the **packages.yml** configuration file. Running **dbt deps** installs the required dependencies, making the macros accessible within the project. Macro behavior can be tailored through parameterization, allowing customization without altering the core logic



Data validation based on role-based access

Access to masked and plaintext data is governed by Snowflake roles. Masking policies enforce the correct view for each role automatically at query time.

Role	Environment	HC Column (Highly Confidential)	Unclassified Column	CONF Column (Confidential)	PRIVATE Column	PUBLIC Column
DEVELOPER ROLES						
Landing Subject Area (HC Tier)	DEV	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext
TESTER ROLES						
Zone Subject Area (HC Tier)	DEV, SYST, PPTE	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext
Zone Subject Area (CONFIDENTIAL Tier)	DEV, SYST, PPTE	Masked	Masked	Plaintext	Plaintext	Plaintext
Zone Subject Area (PUBLIC / Private Tier)	DEV, SYST, PPTE	Masked	Masked	Masked	Plaintext	Plaintext
SUPPORT ROLES						
Zone Subject Area (HC Tier)	DEV, SYST, PPTE, PROD	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext

Zone Subject Area (CONFIDENTIAL Tier)	DEV, SYST, PPTE, PROD	Masked	Masked	Plaintext	Plaintext	Plaintext
Zone Subject Area (PUBLIC / Private Tier)	DEV, SYST, PPTE, PROD	Masked	Masked	Masked	Plaintext	Plaintext
ANALYST ROLES						
Zone Subject Area (HC Tier)	PROD	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext
Zone Subject Area (CONFIDENTIAL Tier)	PROD	Masked	Masked	Plaintext	Plaintext	Plaintext
Zone Subject Area (PUBLIC / Private Tier)	PROD	Masked	Masked	Masked	Plaintext	Plaintext
SERVICE ACCOUNT ROLES						
Airflow Service Role	ALL	Plaintext		Plaintext	Plaintext	Plaintext

Environments

The Strategic Classification & Masking solution will be implemented and tested across both Snowflake accounts following BDH standards - pre-production (DEV/SYST) and production (PPTE/PROD).

Testing

The solution will be tested across four key areas. Tag validation will confirm that **SECURITY_CLASSIFICATION_CODE, BUSINESS_KEY_FLAG, and PRIMARY_KEY_FLAG tags** are correctly applied to all in-scope tables and columns in Snowflake. Masking validation will verify that masking policies are properly attached and behave as expected based on the user's role. Hashing validation will ensure **SHA-256** hashing is applied correctly on all Primary Key and Business Key columns. Finally, an end-to-end test will be executed to validate the full pipeline from Alation classification through to tag application and masking enforcement in Snowflake.

Note :Data products engineering team should define the constraints in **Yaml** file for PK's and test propagation to Snowflake Database.
Then metadata extraction job has to run for Snowflake PreProd source in Alation to ingest . Verify that the PKs appear in Alation UI on the tables page then validate that the columns are reflected as primary keys in Alation Analytics

Strategic Classification & Masking Testcases - Digital, Data & Analytics - BNZ Confluence

Transform Database - Personal Schema Data Protection

As discussed regarding the **Transform database**, no action is required at the moment since this is a **DEV sandbox environment** and is not expected to contain sensitive data. If any exception occurs, the **Zone automation** pipeline will be used as **UBAC**.

Airflow DAG Triggering Guide

DAG ID	scm_macro_scm_apply_tags
Schedule	None (Manual Trigger Only)
dbt Macro	scm_apply_tags

This DAG is a manually-triggered Airflow pipeline that executes the dbt macro scm_apply_tags against a specified Snowflake table. It does not run on any schedule and must be triggered explicitly via the Airflow UI or API with the required configuration parameters

Configuration Parameters

When triggering the DAG, you must provide a JSON configuration in the "Trigger DAG w/ config" screen. The parameters are described below.

Parameter	Type	Required	Description
db_name	String	Yes	Snowflake database name where the table resides
schema_name	String	Yes	Snowflake schema name
table_name	String	Yes	Target table name to apply tags on
since_date	String	No	Filter date in YYYY-MM-DD format. If omitted, macro processes all records (macro default = none)

How to Trigger the DAG

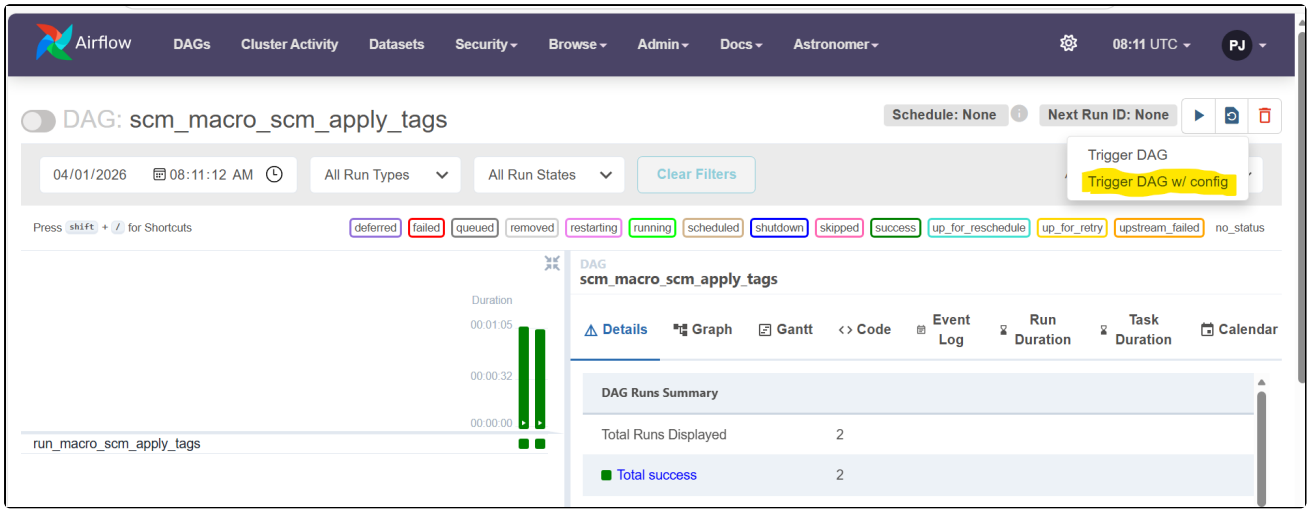
Step 1: Navigate to the DAG

- Log in to the Airflow UI
- Search for scm_macro_scm_apply_tags in the DAGs list
- Click on the DAG name to open it

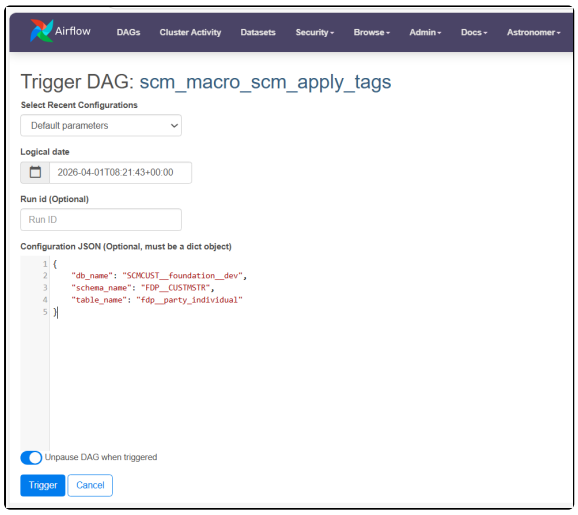
Step 2: Trigger with Config

- Click the Play button (triangle icon) at the top right
- Select "Trigger DAG w/ config" from the dropdown
- A JSON editor will appear — paste your configuration JSON
- Click "Trigger" to start the run

Supported Trigger Scenarios



Configuration JSON Page



Scenario 1 - Without since_date (Process All Records)

Use this when you want to apply tags to all records in the table without any date filter.

```
{
   "db_name":    "SCMCUST__FOUNDATION__DEV",
   "schema_name": "FDP__CUSTMSTR",
   "table_name":  "fdp__party_individual"
}
```

Result: since_date is passed as none to the macro, which uses its default behaviour (no date filter).

Scenario 2 - With since_date (Filter from a Specific Date)

Use this when you want to apply tags only to records updated on or after a specific date.

```
{
   "db_name":     "SCMCUST__FOUNDATION__DEV",
   "schema_name": "FDP__CUSTMSTR",
   "table_name":  "fdp__party_individual",
   "since_date":  "2026-03-28"
}
```

Result: since_date is passed as '2026-03-28' to the macro, which filters records from that date onwards.

Important Notes

- This DAG has no schedule — it will only run when manually triggered.
- All three parameters (db_name, schema_name, table_name) are mandatory. The DAG will fail immediately with a KeyError if any are missing.
- since_date must follow YYYY-MM-DD format. Invalid formats will cause a Snowflake timestamp error.
- Do not pass since_date as an empty string or null text — omit the key entirely if not needed.

DAG Technical Details

For reference, the key configuration in the DAG code is:

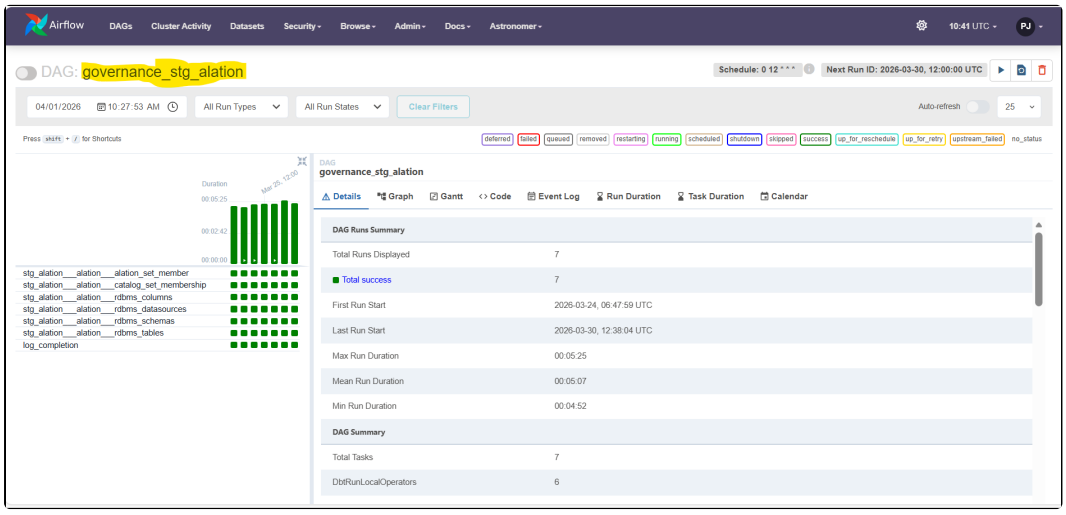
```
# DAG is manually triggered only
schedule_interval=None

# Ensures Jinja renders Python None (not string "None")
render_template_as_native_obj=True

# Args passed to dbt macro
args={
    "db_name":     "{{ dag_run.conf['db_name'] }}",
    "schema_name": "{{ dag_run.conf['schema_name'] }}",
    "table_name":  "{{ dag_run.conf['table_name'] }}",
    "since_date":  "{{ dag_run.conf['since_date'] if 'since_date' in dag_run.conf else none }}",
}
```

DAG: governance_stg_alation

Alation Staging Models - Daily Load

This DAG runs daily and loads data from the Alation source into the GOVERNANCE__STG__ALATION__PREPROD Snowflake database.

It executes 6 dbt staging models sequentially, each configured with incremental load type from a shared JSON config file.



DAG Details

DAG ID	governance_stg_alation
Schedule	0 12 * * * (**Daily at 12:00 PM UTC**)
Trigger Type	Automatic (Scheduled)
Target DB	GOVERNANCE__STG__ALATION__PREPROD
Schema	ALATION

Models Executed

All 6 models run sequentially in the order listed below. Each model reads its load configuration (run_type, days_back, tool_name) from governance_models_loadtype_config.json.

#	Model Name	Target Table
1	stg_alation___alation___alation_set_member	ALATION_SET_MEMBER
2	stg_alation___alation___catalog_set_membership	CATALOG_SET_MEMBERSHIP
3	stg_alation___alation___rdbms_columns	RDBMS_COLUMNS
4	stg_alation___alation___rdbms_datasources	RDBMS_DATASOURCES
5	stg_alation___alation___rdbms_schemas	RDBMS_SCHEMAS
6	stg_alation___alation___rdbms_tables	RDBMS_TABLES

Snowflake Target

Component	Value
Database	GOVERNANCE__STG__ALATION__PREPROD

Schema	ALATION
Tables (6)	ALATION_SET_MEMBER, CATALOG_SET_MEMBERSHIP, RDBMS_COLUMNS, RDBMS_DATASOURCES, RDBMS_SCHEMAS, RDBMS_TABLES

Important Notes

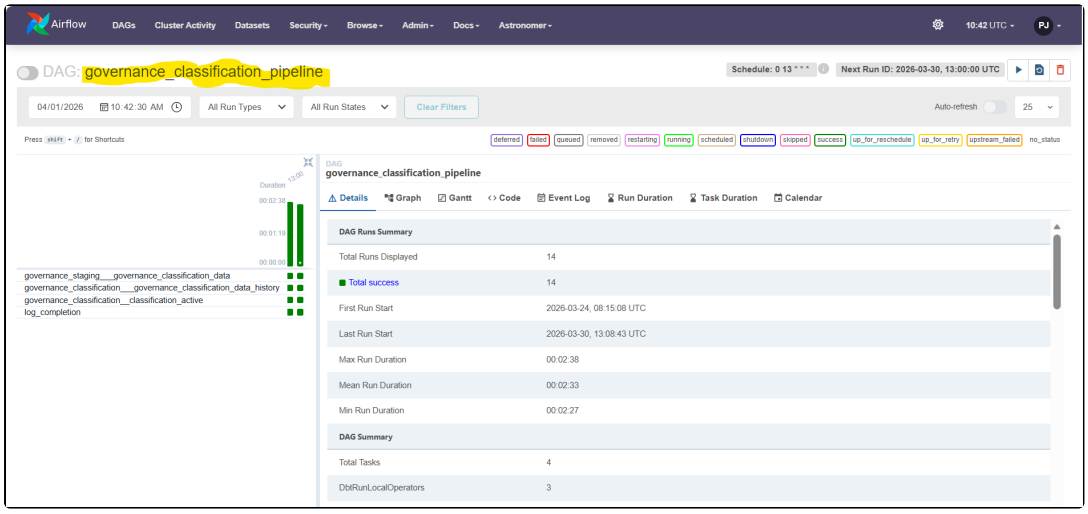
- This DAG runs automatically - no manual trigger needed.
- max_active_runs=1 ensures only one instance runs at a time, preventing duplicate loads.
- All models use incremental load by default - only new/changed records are processed.
- log_completion task at the end confirms all models ran successfully.

DAG: governance_classification_pipeline

Classification Pipeline — Daily Load

This DAG runs daily and executes the governance classification pipeline, loading data into the GOVERNANCE__PREPROD Snowflake database.

It processes 3 dbt models sequentially: first staging, then history, then the active classification view.



DAG Details

DAG ID	governance_classification_pipeline
Schedule	0 13 * * * (**Daily at 1:00 PM UTC**)
Trigger Type	Automatic (Scheduled)
Target DB	GOVERNANCE__PREPROD

Models Executed

All 3 models run sequentially in the strict order below. The full_refresh flag is set per model based on the load_type in governance_models_loadtype_config.json.

#	Model Name	Target
1	governance_staging___governance_classification_data	GOVERNANCE_STAGING.GOVERNANCE_CLASSIFICATION_DATA
2	governance_classification___governance_classification_data_history	GOVERNANCE_CLASSIFICATION. GOVERNANCE_CLASSIFICATION_DATA_HISTORY
3	governance_classification__classification_active	GOVERNANCE_CLASSIFICATION.CLASSIFICATION_ACTIVE (View)

Snowflake Target

Component	Value
Database	GOVERNANCE__PREPROD
Schema (Staging)	GOVERNANCE_STAGING
Schema (Classification)	GOVERNANCE_CLASSIFICATION
Tables	GOVERNANCE_CLASSIFICATION_DATA, GOVERNANCE_CLASSIFICATION_DATA_HISTORY
Views	CLASSIFICATION_ACTIVE

Pipeline Dependency - Why Order Matters

The 3 models must run in strict sequence because each depends on the output of the previous:

Step	Model	Why it must run in this order
1	governance_staging___governance_classification_data	Loads raw classification data into staging. History model depends on this.
2	governance_classification___governance_classification_data_history	Builds historical records from staging data. Active view depends on this.
3	governance_classification__classification_active	Creates the active view from history. Final output for downstream consumers.

Important Notes

- This DAG runs automatically every day at 1:00 PM UTC - runs after governance_stg_alation (12:00 PM).
- full_refresh flag is model-specific - check governance_models_loadtype_config.json to see which models use full load.
- Step order is critical - do not change CLASSIFICATION_FLOW list order without understanding upstream/downstream dependencies.
- CLASSIFICATION_ACTIVE is a View, not a Table - it reflects the latest state from GOVERNANCE_CLASSIFICATION_DATA_HISTORY.

Combined DAG Schedule Summary

Both DAGs are part of the same governance pipeline and are scheduled to run sequentially:

DAG ID	Schedule (UTC)	Target DB	Models
governance_stg_alation	0 12 * * * (12:00 PM)	GOVERNANCE__STG__ALATION__PREPROD	6 staging models
governance_classification_pipeline	0 13 * * * (1:00 PM)	GOVERNANCE__PREPROD	3 classification models

Note: The classification pipeline is scheduled 1 hour after the alation staging DAG to ensure staging data is ready before classification processing begins.

Snowflake Tag Management — UNSET Guide

Purpose: Step-by-step reference for auditing and removing object tags in Snowflake across all levels: Column -> Table -> Schema -> Database.

Overview

In Snowflake, tags can be applied at four object levels. When removing tags, they **must be unset in a specific sequence** - starting from the most granular level (Column) and moving up to the broadest (Database).

Order	Level	Object Type
1 Start here	**COLUMN**	Individual column within a table
2	**TABLE**	Entire table
3	**SCHEMA**	Entire schema
4 End here	**DATABASE**	Entire database

Important: Skipping or reversing this order may result in incomplete tag removal. Always complete all 4 steps.

Step 1 - Audit: Find Existing Tags Before Removal

Before running any UNSET commands, use the queries below to identify all tags currently applied at each level.

Find Tags at COLUMN Level

```
-- Find all column-level tags on a specific table
SELECT  UPPER(COLUMN_NAME)  AS COLUMN_NAME
      ,TAG_NAME
      ,TAG_VALUE
      ,TAG_DATABASE
      ,TAG_SCHEMA
FROM TABLE(
   <DATABASE_NAME>.information_schema.TAG_REFERENCES_ALL_COLUMNS(
      '<DATABASE_NAME>.<SCHEMA_NAME>.<TABLE_NAME>',
      'table'
   )
)
WHERE TAG_NAME IN ('SECURITY_CLASSIFICATION_CODE')
-- To check multiple tags, uncomment and extend the list below:
-- WHERE TAG_NAME IN ('BUSINESS_KEY_FLAG','PRIMARY_KEY_FLAG','SECURITY_CLASSIFICATION_CODE')
;
```

Find ALL Tags Across All Levels (Single Query)

Gives a complete picture across all 4 levels for a given database at once

```
SELECT  DOMAIN
      ,UPPER(OBJECT_NAME)   AS OBJECT_NAME
      ,UPPER(COLUMN_NAME)   AS COLUMN_NAME
      ,TAG_NAME
      ,TAG_VALUE
      ,TAG_DATABASE
      ,TAG_SCHEMA
      ,OBJECT_SCHEMA        AS SCHEMA_NAME
      ,OBJECT_DATABASE      AS DATABASE_NAME
FROM SNOWFLAKE.ACCOUNT_USAGE.TAG_REFERENCES
WHERE OBJECT_DATABASE = '<DATABASE_NAME>'        -- Replace with your DB
-- AND TAG_NAME IN ('SECURITY_CLASSIFICATION_CODE')  -- Optional: filter by tag name
ORDER BY
   CASE DOMAIN
      WHEN 'COLUMN'   THEN 1
      WHEN 'TABLE'    THEN 2
      WHEN 'SCHEMA'   THEN 3
      WHEN 'DATABASE' THEN 4
   END
  ,OBJECT_SCHEMA
  ,OBJECT_NAME
  ,COLUMN_NAME
;
```

Step 2 - Remove Tags Using UNSET (Follow This Sequence)

Once you've identified all tags from Step 1, run the UNSET commands in the order below.

UNSET at COLUMN Level (Start Here)

```
-- Unset a single tag from one column
ALTER TABLE <DATABASE_NAME>.<SCHEMA_NAME>.<TABLE_NAME>
   MODIFY COLUMN <COLUMN_NAME>
   UNSET TAG <TAG_DATABASE>.<TAG_SCHEMA>.<TAG_NAME>;
```

Example:
ALTER TABLE **SCMCUST__FOUNDATION__SYST.FDP__CUSTMSTR.FDP__PARTY_COMMUNICATION** MODIFY COLUMN **DWH_SOURCE_SYSTEM_CODE** UNSET TAG **GOVERNANCE__SYST.GOV_TAGGING.ANCHOR_TAG**, GOVERNANCE__SYST.GOV_TAGGING.CREDENTIALS_IDENTIFIER_FLAG, GOVERNANCE__SYST.GOV_TAGGING.CREDIT_IDENTIFIER_FLAG, GOVERNANCE__SYST.GOV_TAGGING.GOVERNMENT_IDENTIFIER_FLAG, GOVERNANCE__SYST.GOV_TAGGING.HEALTH_IDENTIFIER_FLAG, GOVERNANCE__SYST.GOV_TAGGING.PCI_DSS_COMPLIANCE_IDENTIFIER_FLAG, GOVERNANCE__SYST.GOV_TAGGING.PRIVACY_IDENTIFIER_FLAG, GOVERNANCE__SYST.GOV_TAGGING.SECURITY_CLASSIFICATION_CODE, GOVERNANCE__SYST.GOV_TAGGING.SENSITIVE_PERSONAL_IDENTIFIER_FLAG, GOVERNANCE__SYST.GOV_TAGGING.ZONE_TAG;

UNSET at TABLE Level

```
-- Unset a single tag from a table
ALTER TABLE <DATABASE_NAME>.<SCHEMA_NAME>.<TABLE_NAME>
   UNSET TAG <TAG_DATABASE>.<TAG_SCHEMA>.<TAG_NAME>;
```

Example:

ALTER TABLE **SCMCUST__FOUNDATION__SYST.FDP__CUSTMSTR.FDP__MANAGED_GROUP_PARTY_LINK** UNSET TAG GOVERNANCE__SYST.GOV_TAGGING.ANCHOR_TAG, GOVERNANCE__SYST.GOV_TAGGING.SECURITY_CLASSIFICATION_CODE, GOVERNANCE__SYST.GOV_TAGGING.ZONE_TAG;

UNSET at SCHEMA Level

```
-- Unset a single tag from a schema
ALTER SCHEMA <DATABASE_NAME>.<SCHEMA_NAME>
   UNSET TAG <TAG_DATABASE>.<TAG_SCHEMA>.<TAG_NAME>;
```

Example:

ALTER TABLE **SCMCUST__FOUNDATION__SYST.FDP__CUSTMSTR** UNSET TAG GOVERNANCE__SYST.GOV_TAGGING.ANCHOR_TAG, GOVERNANCE__SYST.GOV_TAGGING.SECURITY_CLASSIFICATION_CODE, GOVERNANCE__SYST.GOV_TAGGING.ZONE_TAG;

UNSET at DATABASE Level (End Here)

```
-- Unset a single tag from a database
ALTER DATABASE <DATABASE_NAME>
   UNSET TAG <TAG_DATABASE>.<TAG_SCHEMA>.<TAG_NAME>;
```

Example:

ALTER DATABASE **SCMCUST__FOUNDATION__SYST** UNSET TAG GOVERNANCE__SYST.GOV_TAGGING.ANCHOR_TAG, GOVERNANCE__SYST.GOV_TAGGING.ZONE_TAG;

Handover Steps: Migration from Interim to Strategic Classification & Masking

Summary of key steps required for Migration

As part of the migration from Interim Governance to Strategic Classification & Masking using Snowflake,dbt,airflow, please follow the below handover steps carefully.

Step 1: Remove Existing Interim Governance Objects

- Identify all tags applied at database, schema, table, and column levels
- Remove masking policies and tags from columns using ALTER statements
- Remove tags from tables, schemas, and databases

Step 2: Import Reusable dbt Components from Platform dbt to Zone dbt

- Import required components from the dbt platform repository using `dbt deps`
- Ensure the following are included:
    - generate_alias_name
    - generate_database_name
    - generate_schema_name
    - scm_apply_tags
    - generic_sources.yml

Step 3: Update dbt Project Configuration in Zone dbt

- Update the `dbt_project.yml` file for the respective zone
- Add the post-hook to invoke tagging macro:

  {{ scm_apply_tags() }}

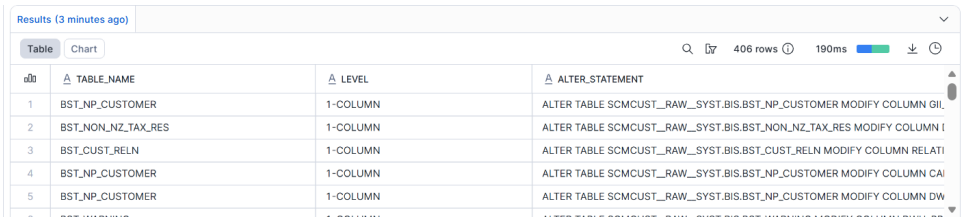
Step 4: Run dbt/Airflow Jobs to apply classifications

Detailed Steps for the key steps summarized above

Step 1 Detailed: Unset Existing Tags (Execution Process)

- Run the query in Snowflake UI to generate ALTER statements for all levels

```
EXECUTE IMMEDIATE $$
DECLARE
   V_TABLE_NAME STRING;
   V_FULL_NAME STRING;
   V_SCH_NAME  STRING;
   V_DB STRING DEFAULT 'SCMCUST__RAW__SYST';
   V_SCH STRING DEFAULT 'BIS';
   C_TABLES CURSOR FOR
      SELECT TABLE_NAME
      FROM SCMCUST__RAW__SYST.INFORMATION_SCHEMA.TABLES
      WHERE TABLE_SCHEMA = 'BIS'
        AND TABLE_TYPE = 'BASE TABLE';
BEGIN
   CREATE OR REPLACE TEMPORARY TABLE TEMP_UNSET_STATEMENTS (
      TABLE_NAME STRING,
      LEVEL STRING,
      ALTER_STATEMENT STRING
   );

   FOR REC IN C_TABLES DO
      V_TABLE_NAME := REC.TABLE_NAME;
      V_FULL_NAME := V_DB || '.' || V_SCH || '.' || V_TABLE_NAME;
      V_SCH_NAME  :=  V_DB || '.' || V_SCH;

       -- SCHEMA-LEVEL TAGS
      INSERT INTO TEMP_UNSET_STATEMENTS
      SELECT
         DISTINCT
         :V_SCH,
         '3-SCHEMA',
         'ALTER TABLE ' || :V_SCH_NAME || ' UNSET TAG ' ||
         LISTAGG(TAG_DATABASE || '.' || TAG_SCHEMA || '.' || TAG_NAME, ', ')
         WITHIN GROUP (ORDER BY TAG_NAME) || ';'
      FROM TABLE(SCMCUST__RAW__SYST.INFORMATION_SCHEMA.TAG_REFERENCES(
         :V_SCH_NAME, 'SCHEMA'
      ))
      WHERE COLUMN_NAME IS NULL
      HAVING COUNT(*) > 0;

      -- TABLE-LEVEL TAGS
      INSERT INTO TEMP_UNSET_STATEMENTS
      SELECT
         :V_TABLE_NAME,
         '2-TABLE',
         'ALTER TABLE ' || :V_FULL_NAME || ' UNSET TAG ' ||
         LISTAGG(TAG_DATABASE || '.' || TAG_SCHEMA || '.' || TAG_NAME, ', ')
         WITHIN GROUP (ORDER BY TAG_NAME) || ';'
      FROM TABLE(SCMCUST__RAW__SYST.INFORMATION_SCHEMA.TAG_REFERENCES(
         :V_FULL_NAME, 'TABLE'
      ))
      WHERE COLUMN_NAME IS NULL
      HAVING COUNT(*) > 0;

      -- COLUMN-LEVEL TAGS
      INSERT INTO TEMP_UNSET_STATEMENTS
      SELECT
         :V_TABLE_NAME,
         '1-COLUMN',
         'ALTER TABLE ' || :V_FULL_NAME ||
         ' MODIFY COLUMN ' || UPPER(COLUMN_NAME) || ' UNSET TAG ' ||
         LISTAGG(TAG_DATABASE || '.' || TAG_SCHEMA || '.' || TAG_NAME, ', ')
         WITHIN GROUP (ORDER BY TAG_NAME) || ';'
      FROM TABLE(SCMCUST__RAW__SYST.INFORMATION_SCHEMA.TAG_REFERENCES_ALL_COLUMNS(
         :V_FULL_NAME, 'TABLE'
      ))
      WHERE COLUMN_NAME IS NOT NULL
      GROUP BY COLUMN_NAME
      HAVING COUNT(*) > 0;
   END FOR;
END;
$$;

SELECT  DISTINCT TABLE_NAME,LEVEL,ALTER_STATEMENT  FROM TEMP_UNSET_STATEMENTS ORDER BY LEVEL ;
```



DataBase

ALTER DATABASE **SCMCUST__RAW__SYST** UNSET TAG GOVERNANCE__SYST.GOV_TAGGING.ANCHOR_TAG, GOVERNANCE__SYST.GOV_TAGGING.ZONE_TAG;

Note: We need to replace the database name and schema for generating alter statements for snowflake objects

```
-- ============================================================================
-- Generate UNSET TAG statements - ONE ALTER per TAG
-- Database: SCMCUST__FOUNDATION__SYST
-- Schema: FDP__CUSTMSTR
-- ============================================================================

-- Step 1: Create temp table to store results
CREATE OR REPLACE TEMP TABLE TEMP_TAG_UNSET_STATEMENTS (
    OBJECT_TYPE STRING,
    OBJECT_NAME STRING,
    TAG_NAME STRING,
    ALTER_STATEMENT STRING
);

-- Step 2: Populate temp table with UNSET statements
EXECUTE IMMEDIATE $$
DECLARE
    V_DB STRING DEFAULT 'SCMCUST__FOUNDATION__SYST';
    V_SCH STRING DEFAULT 'FDP__CUSTMSTR';
    V_SCH_FULL STRING;
    table_list STRING;
    table_name STRING;
    full_table_name STRING;
    sql_cmd STRING;
BEGIN
    V_SCH_FULL := V_DB || '.' || V_SCH;

    -- ========================================================================
    -- SCHEMA-LEVEL TAGS
    -- ========================================================================
    sql_cmd :=
        'INSERT INTO TEMP_TAG_UNSET_STATEMENTS ' ||
        'SELECT ' ||
        '    "SCHEMA", ' ||
        '    '' || V_SCH_FULL || '', ' ||
        '    TAG_DATABASE || "." || TAG_SCHEMA || "." || TAG_NAME, ' ||
        '    "ALTER SCHEMA ' || V_SCH_FULL || ' UNSET TAG " || ' ||
        '    TAG_DATABASE || "." || TAG_SCHEMA || "." || TAG_NAME || ";" ' ||
        'FROM TABLE(SCMCUST__FOUNDATION__SYST.INFORMATION_SCHEMA.TAG_REFERENCES(' ||
        '    '' || V_SCH_FULL || '', "SCHEMA" ' ||
        ')) ' ||
        'WHERE COLUMN_NAME IS NULL';
    EXECUTE IMMEDIATE sql_cmd;

    -- ========================================================================
    -- TABLE and COLUMN-LEVEL TAGS
    -- ========================================================================
    DECLARE
        c1 CURSOR FOR
            SELECT TABLE_NAME
            FROM SCMCUST__FOUNDATION__SYST.INFORMATION_SCHEMA.TABLES
            WHERE TABLE_SCHEMA = 'FDP__CUSTMSTR'
              AND TABLE_TYPE = 'BASE TABLE';
    BEGIN
        FOR record IN c1 DO
            table_name := record.TABLE_NAME;
            full_table_name := V_DB || '.' || V_SCH || '.' || table_name;
```

```
            -- TABLE-LEVEL TAGS (one row per tag)
            sql_cmd :=
               'INSERT INTO TEMP_TAG_UNSET_STATEMENTS ' ||
               'SELECT ' ||
               '    "TABLE", ' ||
               '    "' || full_table_name || '", ' ||
               '    TAG_DATABASE || "." || TAG_SCHEMA || "." || TAG_NAME, ' ||
               '    "ALTER TABLE ' || full_table_name || ' UNSET TAG " || ' ||
               '    TAG_DATABASE || "." || TAG_SCHEMA || "." || TAG_NAME || ";" ' ||
               'FROM TABLE(SCMCUST__FOUNDATION__SYST.INFORMATION_SCHEMA.TAG_REFERENCES(' ||
               '    "' || full_table_name || '", "TABLE" ' ||
               ')) ' ||
               'WHERE COLUMN_NAME IS NULL';
            EXECUTE IMMEDIATE sql_cmd;

            -- COLUMN-LEVEL TAGS (one row per tag per column)
            sql_cmd :=
               'INSERT INTO TEMP_TAG_UNSET_STATEMENTS ' ||
               'SELECT ' ||
               '    "COLUMN", ' ||
               '    "' || full_table_name || '" || "." || COLUMN_NAME, ' ||
               '    TAG_DATABASE || "." || TAG_SCHEMA || "." || TAG_NAME, ' ||
               '    "ALTER TABLE ' || full_table_name || ' MODIFY COLUMN " || COLUMN_NAME || ' ||
               '    " UNSET TAG " || TAG_DATABASE || "." || TAG_SCHEMA || "." || TAG_NAME || ";" ' ||
               'FROM TABLE(SCMCUST__FOUNDATION__SYST.INFORMATION_SCHEMA.TAG_REFERENCES_ALL_COLUMNS(' ||
               '    "' || full_table_name || '", "TABLE" ' ||
               ')) ' ||
               'WHERE COLUMN_NAME IS NOT NULL';
            EXECUTE IMMEDIATE sql_cmd;
         END FOR;
      END;

   END;
   $$;

   -- ============================================================================
   -- RESULTS
   -- ============================================================================

   -- Summary by object type
   SELECT
      OBJECT_TYPE,
      COUNT(*) AS STATEMENT_COUNT
   FROM TEMP_TAG_UNSET_STATEMENTS
   GROUP BY OBJECT_TYPE
   ORDER BY OBJECT_TYPE;

   -- Detailed view with object names
   SELECT
      OBJECT_TYPE,
      OBJECT_NAME,
      TAG_NAME,
      ALTER_STATEMENT
   FROM TEMP_TAG_UNSET_STATEMENTS
   ORDER BY
      CASE OBJECT_TYPE
         WHEN 'COLUMN' THEN 1
         WHEN 'TABLE' THEN 2
         WHEN 'SCHEMA' THEN 3
      END,
      OBJECT_NAME,
      TAG_NAME;

   -- Copy-paste ready: Just the ALTER statements
   SELECT ALTER_STATEMENT
   FROM TEMP_TAG_UNSET_STATEMENTS
   ORDER BY
      CASE OBJECT_TYPE
         WHEN 'COLUMN' THEN 1
         WHEN 'TABLE' THEN 2
         WHEN 'SCHEMA' THEN 3
      END,
      OBJECT_NAME,
      TAG_NAME;
   ```
   DataBase
   ALTER DATABASE **SCMCUST__RAW__SYST** UNSET TAG GOVERNANCE__SYST.GOV_TAGGING.ANCHOR_TAG,
   GOVERNANCE__SYST.GOV_TAGGING.ZONE_TAG;

   Note: We need to replace the database name and schema for generating alter statements for snowflake objects
- Copy the generated statements into a .sql file
- Execute the script using governance Flyway
- Validate that all existing tags and masking policies are removed

Step 4 Detailed: Apply Strategic Classification & Masking

- Ensure Steps 2 and 3 are completed as mentioned in Summary above
- Trigger the Airflow DAG for the respective zone/model
    - Example: Zone: SCMCUST, Table: FDP__PARTY_CORE
- Verify that classification tags and masking policies are applied successfully

Validation Checklist

- Interim tags and masking policies removed.
- dbt dependencies installed successfully.
- dbt_project.yml updated with post-hook.
- Flyway execution completed without errors.
- Airflow DAG executed successfully.
- Strategic classification and masking applied correctly.

Governance zone Access Request and URL's

<table>
<tr><th>Tool</th><th>AD Groups</th><th>url</th></tr>
<tr><td>Github</td><td>A_BDH_github_fg_governance_admin
A_BNZ_Git_dap_bdh_governance_committers</td><td>
<table>
<tr><th>Repo name</th><th>URL</th><th>Preprod</th><th>Prod</th></tr>
<tr><td>bdh_governance_dbt_repo</td><td>bnz-dap/bdh_governance_dbt_repo</td><td>Branch :Master</td><td></td></tr>
<tr><td>bdh_governance_sfobj_repo</td><td>bnz-dap/bdh_governance_sfobj_repo</td><td>Branch :Master

Executable Folder :
BDH_MIGRATION</td><td></td></tr>
<tr><td>bdh_governance_dbt_gitsync_repo</td><td>bnz-dap
/bdh_governance_dbt_gitsync_repo</td><td>Branch:dev</td><td>Branch :Prod</td></tr>
</table>
</td></tr>
<tr><td>Astro</td><td>A_BDH_astro_fg_governance_dev_viewer
A_BDH_astro_fg_governance_prod_viewer</td><td>
<table>
<tr><th>Env</th><th>Work Space Name</th><th>URL</th></tr>
<tr><td>Preprod</td><td>airflow-bdh-uplift-governance-dev</td><td>https://app.ppte.astro.nz.thenational.com/</td></tr>
<tr><td>Prod</td><td>airflow-bdh-uplift-governance-prod</td><td>https://app.prod.astro.nz.thenational.com/</td></tr>
</table>
</td></tr>
<tr><td>Snowflake DB</td><td>A_BDH_sf_fg_governance_dev_developer
A_BDH_sf_fg_governance_dev_consumer
A_BDH_sf_fg_governance_syst_consumer
A_BDH_sf_fg_governance_ppte_consumer
A_BDH_sf_fg_governance_prod_consumer</td><td>
<table>
<tr><th>Env</th><th>Share DB</th><th>Alation DB</th><th>Classification DB</th><th>Logging DB</th></tr>
<tr><td>Preprod</td><td>ALATION__ANALYTICS__SHARE</td><td>GOVERNANCE__STG_ALATION__PREPROD</td><td>GOVERNANCE__PREPROD</td><td>GOVERNANCE__LOGGING__PREPROD</td></tr>
<tr><td>Prod</td><td></td><td>BNZ_DAP_PREPROD_AWS_GOVERNANCE__STG_ALATION__PREPROD_SNOWFLAKE_SHARE</td><td>GOVERNANCE__PROD</td><td>GOVERNANCE__LOGGING__PROD</td></tr>
</table>
</td></tr>
<tr><td>Jenkins</td><td>A_Managedjenkins_bdh-governance_users</td><td>https://jenkins-bdhgovernance.ci.bnz.digital/

Sync job: All [GitHub Bitbucket Sync Jobs] - Jenkins
JTE Build Job : All [JTE Build] - Jenkins</td></tr>
</table>

Data validation CUST - SCMCUST (PRE and POST) Results

SCM_validatin_pre_post.xlsx

Conclusion

This solution provides a scalable, automated, and governance-driven framework leveraging Alation Data Share and Snowflake capabilities to enforce consistent data protection policies across the organization.

Reference's

BDH Classification & Masking Design - Digital, Data & Analytics - BNZ Confluence